Práctica de Estructuras Lineales

Todos los algoritmos planteados deben ser desarrollados utilizando el paradigma orientado a objetos.

1. Implemente un constructor polimórfico para la clase Nodo, el cual tenga como parámetros el elemento y el apuntador al siguiente nodo.

2. Realice los algoritmos correspondientes a los métodos consultar, eliminar y destruir de la segunda representación de lista enlazada.

3. Extienda la clase Lista (simplemente enlazada) con los siguientes métodos:
   a. **proc** modificar(Elemento: e; **Entero**: posicion)
   b. **proc** insertarAlPrincipio(Elemento: e), inserta un elemento al principio de la lista.
   c. **proc** insertarAlFinal(Elemento: e), inserta un elemento al final de la lista.
   d. **proc** invertir(), invierte la lista. Debe tener complejidad máxima de O(n).

4. Implemente la clase NodoDoble (no olvide los métodos anterior() y modificarAnterior()).

5. Implemente las clases ListaDE, ListaCircular y ListaCircularDE.

6. Implemente el método destruir de las clases Pila y Cola.

7. Implemente:
   a. Una operación que invierta una Pila usando una Cola.
   b. Una operación que invierta una Cola usando una Pila.

8. Implemente la clase Cola como especialización de la clase Lista.

9. Implemente la función sublista(**Entero:** i, j): Lista, que obtenga una nueva lista con los nodos desde la posición *i* hasta la posición *j*.

Ejercicios de Listas, Pilas y Colas

Para los siguientes ejercicios se cuenta con las clases Lista, Pila y Cola con los siguientes métodos:

Lista	Pila	Cola
proc construir() **func** esVacia(): **Lógico** **proc** insertar(Elemento: e; **Entero**: pos) **proc** eliminar(**Entero**: pos) **func** consultar(**Entero**: pos): Elemento **proc** modificar(Elemento: e; **Entero**: pos) **proc** destruir()	**proc** construir() **func** esVacia(): **Lógico** **proc** apilar(Elemento: e) **proc** desapilar(**Entero**: pos) **func** tope(**Entero**: pos): Elemento **proc** destruir()	**proc** construir() **func** esVacia(): **Lógico** **proc** encolar(Elemento: e) **proc** desencolar(**Entero**: pos) **func** frente(**Entero**: pos): Elemento **proc** destruir()

Puede implementar los métodos que considere necesarios. No puede utilizar apuntadores para sus soluciones.

10. Una forma simple de codificar un mensaje es intercambiar algunos de los elementos que lo componen. En este sentido se desea que usted desarrolle un algoritmo que intercambie entre sí algunos de los caracteres de un mensaje. Informalmente, la codificación propuesta cambia de lugar solamente los caracteres de posición par en el mensaje de la forma siguiente: el carácter de posición par que está más a la izquierda se intercambia con el carácter de posición par que está más a la derecha, el segundo carácter de posición par de más a la izquierda con el segundo de posición par de más a la derecha, etc., hasta llegar al final. La siguiente tabla muestra ejemplos de mensajes iniciales y su codificación:

Mensaje original	**Mensaje codificado**
"bien"	"bnei"
"bien estamos"	"bsem tseanoi"
"bien, estamos"	"boea,se tnmis"
"estamos bien y con provisiones"	"estnmisiboep n coy nriv sooaes"

Asuma que el mensaje está almacenado en una lista de caracteres.

11. Generar los números de la suerte con ayuda de una cola que contiene inicialmente la secuencia desde 1 hasta N. Los números de la suerte se obtienen eliminando de la cola inicial uno de cada dos números (el primero, el tercero, el quinto, etc.). De la cola resultante se elimina un número de cada tres (el primero, el cuarto, el séptimo, etc.), y así sucesivamente hasta que el número de elementos contenidos en la cola sea menor que la distancia de separación entre cada par de números. Desarrolle un algoritmo que dado un número m, el cual representa la distancia máxima de separación entre cada par de números a ser seleccionados, liste los números de la suerte resultantes de aplicar dicho algoritmo. Ejemplo:

m = 4

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	Lista Original
~~1~~ 2 ~~3~~ 4 ~~5~~ 6 ~~7~~ 8 ~~9~~ 10 ~~11~~ 12 ~~13~~ 14 ~~15~~ 16 ~~17~~ 18 ~~19~~ 20	Elementos a Eliminar (Cada 2 posiciones)
2 4 6 8 10 12 14 16 18 20	Lista después de la eliminación
~~2~~ 4 6 ~~8~~ 10 12 ~~14~~ 16 18 ~~20~~	Elementos a Eliminar (Cada 3 posiciones)
4 6 10 12 16 18	Lista después de la eliminación
~~4~~ 6 10 12 ~~16~~ 18	Elementos a Eliminar (Cada 4 posiciones)
6 10 12 18	Lista después de la eliminación

Nota: Puede utilizar solo una estructura auxiliar del tipo pila o cola en caso de ser necesario.

12. Considere un empleado en una ventanilla de atención al público, el cual recibe planillas de clientes o usuarios que llegan a la ventanilla. A cada planilla recibida, se le coloca la fecha y hora de recepción. Una vez recibidas todas las planillas, estas deben ser procesadas para luego ser devueltas a los clientes. Para hacer más rápido el proceso de entrega a los usuarios, el empleado ha decidido organizarlas de tal manera que todas las planillas de clientes cuyo apellido comience por la misma letra estén juntas, pero que se mantengan el orden de llegada. Una solución manual a este problema puede ser la siguiente:

Cuando recibe una planilla, el empleado le coloca la fecha y hora de recepción y la apila junto con las otras planillas recibidas en un único grupo. Una vez finalizado el proceso de recepción, el empleado distribuye las planillas en 26 grupos diferentes de acuerdo a la primera letra del apellido del cliente. Finalizado este proceso, se recorren los 26 grupos alfabéticamente para colocar de nuevo todas las planillas juntas, agrupadas por la primera letra del apellido y conservando en cada grupo el orden de llegada.

Desarrolle un algoritmo que simule el procedimiento manual descrito anteriormente. Usted debe definir el tipo de estructura más adecuada (lista, pila, cola) para implementar los 26 grupos intermedios y el grupo final, de tal manera que se satisfagan los requerimientos del problema.

Asuma que dispone de las siguientes funciones:
fecha → Cadena (fecha actual del sistema) hora → Cadena (hora actual del sistema)

Considere que dispone de la siguiente clase:

```
clase Planilla
    atributos:
        privado:
            Cadena: nombre, apellido, fechaRec, horaRec
    métodos:
        publico:
            proc construir(Cadena: nombre, apellido, fechaRec, horaRec)
            func getNombre(): Cadena
            func getApellido(): Cadena
            func getFechaRec(): Cadena
            func getHoraRec(): Cadena
fclase
```

13. Una cola medieval se comporta como una cola ordinaria, con la única diferencia de que los elementos almacenados en ella se dividen en dos estamentos: nobles y plebeyos. Dentro de cada estamento, los elementos deben ser atendidos en orden de llegada; además, siempre que haya nobles en la cola, estos deben ser atendidos antes que los plebeyos. Implemente la clase ColaMedieval con las siguientes operaciones:

- Crear una cola medieval vacía.
- Consultar si la cola medieval está vacía.
- Insertar un nuevo elemento.
- Eliminar el elemento al frente.
- Consultar el elemento al frente.
- Consultar el número de nobles.
- Consultar el número de plebeyos.
- Destruir la cola medieval.

a. En términos de colas ordinarias.
b. Utilizando memoria dinámica (en este apartado se pueden usar apuntadores).

Observaciones:
Se cuenta con la clase Persona, en la cual existe un método esNoble: → Lógico.
En los apartados a y b las implementaciones de todas las operaciones (menos el destructor) deben tener un coste constante O(1).